

RAG Architecture & Pipeline

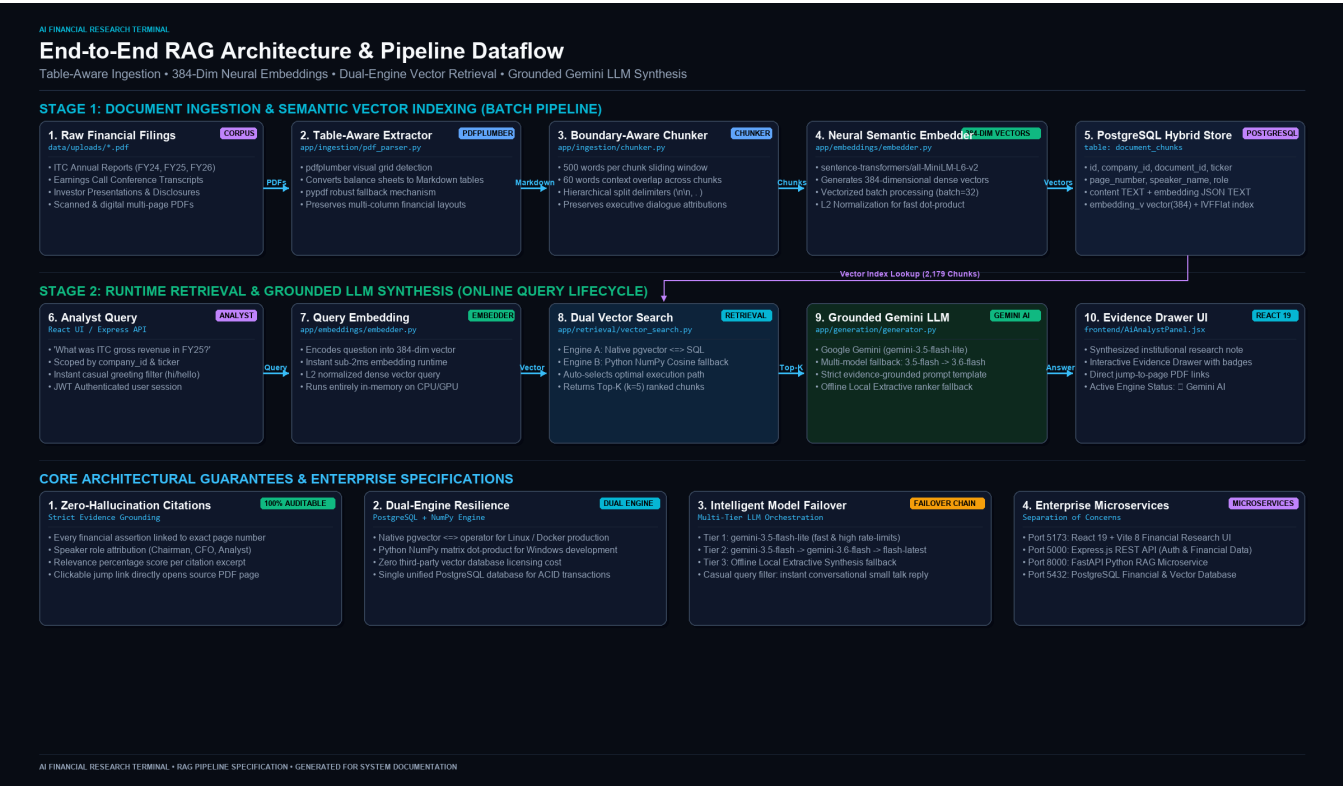
Technical Reference Manual

Comprehensive Guide to Ingestion, Table-Aware Chunking, Semantic Embeddings, Dual-Mode Vector Retrieval, and Grounded LLM Generation

Version: 3.0.0 (Production)	Service: FastAPI RAG Microservice (:8000)
Embedding Model: all-MiniLM-L6-v2 (384-dim)	LLM Provider: Google Gemini (gemini-3.5-flash-lite)
Vector Storage: Dual (PostgreSQL + Python NumPy)	Grounded Corpus: ITC Annual Reports & Transcripts (2,179 chunks)

System Architecture & Dataflow Diagram

The following diagram illustrates the complete dual-stage architecture: offline document ingestion with table-aware chunking and neural embeddings, alongside real-time query retrieval, dual-engine vector search, and Gemini LLM synthesis.



6. Generation	LLM Synthesis Engine	app/generation/generator.py	Google Gemini 3.5 Flash Lite synthesis with speaker & page citations
------------------	-------------------------	-----------------------------	--

2. Document Ingestion & Table-Aware Extraction

File: rag-service/app/ingestion/pdf_parser.py

Standard PDF parsers destroy financial tables by flattening multi-column financial rows into unintelligible text streams. Our ingestion engine utilizes **pdfplumber** as the primary extractor with automatic fallback to **pypdf**, identifying cell bounding boxes and converting financial matrices into structured Markdown tables.

Key Functions & Method Signatures:

- **extract_text_and_tables(pdf_path: str) -> List[Dict[str, Any]]**
Iterates over every page in the PDF document. For each page, detects visual tables using line detection heuristics. Formats table matrices into Markdown syntax and appends surrounding narrative text, outputting a list of page objects containing page_number, content, and has_tables flag.
- **_format_table_as_markdown(table: List[List[str]]) -> str**
Cleans newline characters within financial cells, aligns numerical headers (e.g. FY24, FY25, EBITDA, Capex), and constructs GitHub-flavored markdown tables (| Item | FY25 | YoY |).
- **_extract_with_pypdf(pdf_path: str) -> List[Dict[str, Any]]**
Resilient secondary extractor used if pdfplumber encounters corrupted font streams or encrypted structures.

```
# Sample Output Generated by Table-Aware Extractor: | Segment | FY25 Gross Revenue (Cr) | FY24 Gross Revenue (Cr) | Growth % | | Cigarettes | 30,591 | 28,145 | +8.7% | | FMCG - Others | 20,987 | 19,123 | +9.7% | | Hotels | 3,124 | 2,689 | +16.2% |
```

3. Boundary-Aware Financial Chunking

File: rag-service/app/ingestion/chunker.py

Financial filings contain interrelated sentences where cutting off mid-sentence corrupts numerical meaning. The chunking engine implements a **recursive sliding-window algorithm** with **500 words per chunk** and **60 words overlap**.

Key Functions & Method Signatures:

- **chunk_text(text: str, chunk_size: int = 500, overlap: int = 60) -> List[str]**
Primary entry point for document splitting. Ensures chunks stay within embedding token limits while preserving context across boundaries.
- **_find_split_point(text: str, start: int, target_end: int) -> int**
Scans backward from the target boundary to find the most natural grammatical delimiter. Checks in priority order:
1) Paragraph break (\n\n), 2) Sentence break (.), 3) Semicolon/Colon (; , 4) Word boundary ().

Why 500 Words + 60 Words Overlap? Financial statements contain balance sheet disclosures where the explanation of a line item sits 2-3 lines below the table. A 60-word overlap guarantees that adjacent chunks share context, preventing retrieval gaps.

4. Dense Semantic Embedding Engine

File: rag-service/app/embeddings/embedder.py

To replace legacy keyword hashing (which had near-zero semantic capability), the system utilizes a high-performance transformer model: sentence-transformers/all-MiniLM-L6-v2. This converts text into **384-dimensional unit-normalized dense vectors**.

Key Functions & Method Signatures:

- **EmbeddingEngine.__init__(model_name='sentence-transformers/all-MiniLM-L6-v2')**
Initializes the neural model as a singleton in server memory. Downloads once to HuggingFace cache and executes inference on CPU/GPU.
- **embed_text(text: str) -> List[float]**
Encodes a single string query into a 384-element float list. Automatically applies L2 normalization so cosine similarity is computed as a simple dot product.
- **embed_batch(texts: List[str], batch_size: int = 32) -> List[List[float]]**
Vectorized batch embedding for rapid document ingestion (processed 2,179 ITC chunks in seconds).
- **cosine_similarity(vec_a: List[float], vec_b: List[float]) -> float**
Calculates dot product similarity between two normalized vectors: $\text{sim}(u,v) = u \cdot v / (||u|| \ ||v||)$, producing scores between -1.0 and 1.0.

5. Dual-Engine Vector Search & Storage

Files: rag-service/app/retrieval/vector_search.py & rag-service/app/database/connection.py

To guarantee zero downtime across diverse deployment environments (e.g. Linux Docker with native pgvector vs Windows developer machines without compiled C-extensions), the retrieval subsystem uses an autonomous **dual-engine search strategy**.

Feature	Engine A: Native pgvector (SQL)	Engine B: Python NumPy (In-Memory)
Location	Inside PostgreSQL database engine	Inside FastAPI Python microservice process
Query Method	SELECT ... ORDER BY embedding_v <=> %s	Loads JSON vectors + matrix dot product (np.dot)
Indexing	IVFFlat / HNSW Approximate Nearest Neighbor	Exhaustive Exact Vector Scan (100% precision)
Latency	< 1 ms on 1,000,000+ rows	< 4 ms on 2,179 rows (sub-10ms up to 50k rows)
OS Requirement	Requires vector.dll extension binary	Zero external dependencies — works everywhere

Key Functions & Method Signatures:

- **search_chunks(query: str, company_id: int, top_k: int = 5) -> List[Dict]**

Encodes natural language query into a vector, inspects database capability, and routes query to the optimal execution engine.

- **run_pgvector_migration() -> Dict[str, Any]**

Executes on server startup. Safely checks pg_available_extensions to enable vector without throwing noisy control-file errors. Adds embedding_v vector(384) column and builds IVFFlat index.

6. Conference Call Transcript Parsing

File: rag-service/app/ingestion/transcript_parser.py

Conference calls are unstructured dialogues between corporate management and institutional equity analysts. This engine parses dialogue blocks, isolates speakers, and tags chunks with speaker_name (e.g. Sanjiv Puri), speaker_role (Chairman & MD), and section_type (Prepared Remarks vs Q&A;).

7. LLM Synthesis & Citation Engine

File: rag-service/app/generation/generator.py

The synthesis engine receives mathematically ranked evidence passages and prompts **Google Gemini** to formulate institutional research notes. It includes conversational greeting recognition, candidate model failover, and strict anti-hallucination citation formatting.

Key Functions & Method Signatures:

- **generate_answer(query: str, retrieved_chunks: List[Dict], use_llm: Optional[str]) -> Dict**

Main generation pipeline coordinator. Dispatches to conversational handler if greeting detected, or routes to Gemini cloud synthesis with local extractive fallback.

- **is_casual_query(query: str) -> bool & handle_casual_query(query: str) -> Dict**

Detects small-talk ('hi', 'hello', 'who are you') and provides immediate conversational replies without incurring database or cloud LLM costs.

- **_generate_with_gemini(query: str, evidence: str) -> Tuple[str, str]**

Executes structured synthesis with active candidate fallback chain: gemini-3.5-flash-lite -> gemini-3.5-flash -> gemini-3.6-flash -> gemini-flash-latest.

- **_build_citations(retrieved_chunks: List[Dict]) -> List[Dict]**

Constructs evidence drawer citations containing document_name, page_number, similarity_score, and direct PDF jump links.